System Design: Consistency

English study notes based on the supplied Hindi YouTube lecture (about 6 minutes). Includes examples, consistency models, trade-offs and interview revision.

1. What is consistency?

Consistency describes what values clients observe when data is read or updated, especially when the system maintains multiple copies (replicas). In the lecture, the goal is that users should not see conflicting versions of important data.

Bank example: Your balance is Rs. 0. You deposit Rs. 100, but an ATM still shows Rs. 0 because its data has not yet reflected the deposit. This illustrates a stale read and a potential consistency problem.

Movie ticket example: A customer in Delhi books a seat, while another customer in Pune reads an out-of-date copy of the seat inventory and tries to book the same seat. The booking system must coordinate writes to prevent double booking.

2. Monolithic vs distributed systems

Single-instance/centralized setup: When all reads and writes use one authoritative database, keeping one view of data is simpler. A monolith is not automatically consistent if it uses multiple replicas or caches.

Distributed setup: Multiple servers or databases may hold replicas. A write accepted by one replica can take time to reach the others, creating a temporary window where reads may return different values.

Lecture model: If two replicas take t1 and t2 to catch up, the last replica catches up after max(t1, t2), assuming the updates complete successfully. This is a simplified illustration, not a universal bound.

3. Ways to improve consistency (from the lecture)

Faster network: Lower network delay can help replicas synchronize sooner.

Restrict reads until updates are visible: Read gating or routing reads to an up-to-date authoritative node can avoid stale results, but may increase latency or reduce availability.

Keep replicas geographically closer: Shorter network paths can reduce propagation delay, though geographic distribution also provides resilience and local performance.

4. Three consistency models

Model	What it means	Example / trade-off
Strong	Reads reflect the required latest completed write under the chosen strong model.	Seat inventory, critical balance reads; coordination can cost latency/availability.
Eventual	If updates stop and replicas remain healthy, copies converge over time; some reads may be stale meanwhile.	Social feed updates; typically improves availability/performance.
Weak	No general guarantee that a read immediately sees the latest write; guarantees depend on the specific model.	Some non-critical, best-effort data flows.

5. Practical design decisions

Ticket booking: Enforce a single winner for a seat using a transaction, uniqueness constraint, conditional write, or equivalent coordination. Strong reads alone are not enough to prevent concurrent double booking.

Banking: Financial transactions need correct, atomic updates and reliable authorization; the exact consistency guarantee depends on the operation.

Social media: A new post taking a short time to appear on a follower feed may be acceptable, so eventual consistency is often suitable.

6. Important terminology corrections

Stale read vs dirty read: The lecture calls a lagging replica read a “dirty read.” More precisely, reading an old committed value is a **stale read**. A **dirty read** means reading an uncommitted change from another transaction.

Strong consistency: It does not always require stopping all reads until every replica is updated. Systems can use leaders, quorums, or coordination protocols to give strong guarantees.

Weak consistency: It does not mean replicas never synchronize; it means the system offers weaker visibility/order guarantees.

7. Interview questions and quick answers

Q1. What is consistency in distributed systems? A guarantee about which versions of shared data clients can observe across nodes.

Q2. Why do replicas create consistency challenges? Writes propagate over networks, and different nodes may briefly have different versions.

Q3. Strong vs eventual consistency? Strong consistency provides stricter read visibility; eventual consistency allows temporary stale data but converges when writes stop.

Q4. How would you avoid double booking? Use atomic reservation logic with a database constraint or conditional transaction, and coordinate concurrent requests.

Q5. How can we reduce stale reads? Use leader reads, quorum strategies, read-after-write guarantees, or wait for replication where appropriate.

8. Quick revision

Consistency = visibility of updates. Replication lag can produce stale reads. Strong, eventual and weak models offer different guarantees. Choose based on business impact: seats and money require stricter correctness; feeds can often tolerate delays.

Source note: The provided auto-generated transcript contains speech-recognition errors. These notes preserve its main topics and examples, while clearly labeling technical clarifications.